

Composing multiple control policies using RL for Turtlebot4 traversal

Mahima Malhotra (0033407989), Jiahui Huang (0033571931)

Index Terms—PD, Robot Learning, Reinforcement Learning, Robot Control, Motion Planning

I. INTRODUCTION

THIS project aims to combine multiple PD control policies to control a TurtleBot's traversal through various 3D environments. The traversal paths that our TurtleBot follows will combine straight lines and various tightness of curvature. Each unique traversal situation will have an individual controller. These controllers are then composed together using Hierarchical Reinforcement Learning (HRL). The purpose of Hierarchical Reinforcement Learning is to break complex tasks, in this case, traversing a complex path, into more manageable subtasks. As such, we train the HRL to utilize the PD policy tailored to a section of the path.

II. METHODS

A. PD Controllers

We tuned 4 PD controllers to steer our TurtleBot, according to various curvatures, based on the heading error of the TurtleBot.

The heading error is the difference between the direction our TurtleBot is facing and the direction the TurtleBot needs to face to reach a horizon point on our planned trajectory. Here, the horizon point (x_h, y_h) is a point on the reference path h points beyond the point on the path closest to our robot located at (x, y) . The heading error and subsequent new heading are calculated as follows:

$$\begin{aligned} 1: dx &\leftarrow x_h - x \\ 2: dy &\leftarrow y_h - y \\ 3: \theta_{\text{head_diff}} &\leftarrow \arctan\left(\frac{dy}{dx}\right) \\ 4: \theta_{\text{head_err}} &\leftarrow \theta_{\text{head_diff}} - \theta_{\text{curr_head}} \\ 5: \theta_{\text{new_head}} &\leftarrow k_p^{\text{head}} \theta_{\text{head_err}} + k_d^{\text{head}} d\theta/dt \end{aligned}$$

B. Path Construction

For training, we constructed paths from piecewise functions consisting of our four baseline curvatures. Each segment of the piecewise function has a random length within a certain range, and segments can start at any point within the given curvature. Specifically, any sine wave section could start at any point of the wave. Initially, for Milestone 2, we performed HRL on composed paths consisting of only two of our four baseline path types, line and medium sine, to test the feasibility of

our design. Afterwards, we scaled up to training with paths composed of all 4 baseline paths. When generating these training paths, we added a layer of randomness related to the order in which path types appeared; however, we limited it such that one path type would not appear twice in a row. Additionally, the length of a composed path scales with the number of path types it consists of.

C. Hierarchical Reinforcement Learning

Utilizing the tuned PD controllers, we trained HRL Models to choose the best PD controller based on the robot's current success and the upcoming path.

To explain how we went about training our model, we will go through how a single episode is run, how a path iteration is run, and how these combine with the path generation to train our model.

We define an episode as training on a path segment with a defined start and goal.

We define a path iteration as the training on various path segments (episodes) taken from a single large paths.

Given these definitions, we will further elaborate on the details of our training methodology.

1) *HRL on One Episode*: To train an episode for all of our models, the underlying logic for running a single episode is as follows:

```

1: envState ← getEnvInfo()
2: step ← 0
3: while notAtGoal do
4:   envState ← getEnvInfo()
5:   pdController ← model.getAction(envState)
6:   reward ← runKSteps(pdController)
7:   loss ← calculateLoss()
8:   model.update(loss)
9: end while

```

Basically, every k-steps, we get the environment information and pass this to the model. The model uses that information to choose which PD controller to utilize. We then run k-steps with the chosen PD controller, calculate the loss, and update the model.

The Environment information that the model takes in is

- 1) Cross-Track Error (CTE)

CTE = $\sqrt{(r_y - y_p)^2 + (r_x - x_p)^2}$ where the robot is located at (r_x, r_y) and the nearest point on the path is (x_p, y_p)

2) Heading Error

Heading Error = $\arctan \frac{h_y - y}{h_x - x}$ where (h_x, h_y) is a point on the horizon and (x, y) is the robot's location.

3) Curvature of Path

$$\text{Curvature} = \frac{\psi_i - \psi_{i+h}}{\text{distance}(i, i+h)}$$

where ψ_n is the heading at the n th point on the path. The point i is the point closest to the robot and the $i+h$ is the horizon point. The distance function calculates the euclidean distance between the provided path points.

4) Robot Velocity

Provided by the robot.

5) Change in CTE

$$\frac{\partial \text{CTE}}{\partial t} = \frac{\text{cte} - \text{prev_cte}}{dt}$$

where $dt = 0.05$.

6) Lookahead error

lookahead err = $-\sin(\psi) * (h_x - x) + \cos(\psi) * (h_y - y)$ where ψ is the robot's current yaw, (h_x, h_y) is a point on the horizon, and (x, y) is the robot's location.

7) Previous PD Choice

2) *HRL on a path iteration*: As stated, we define a path iteration as the training on various path segments (episodes) taken from a single large path.

In our training, in a single path iteration, the large path that we utilize is generated via our random construction methodology. Then, we select n random path segments on this generated path. To do so, we randomly select a start point on the path, and the end point is 20% of the overall path away. We choose the random path in this manner to ensure that it is feasible for the robot to reach the end of the path, or the goal, within the maximum step allowance that we provide it. Then, we train on all n paths using our episode training methodology as described.

3) *Total HRL Methodology*: In total, we utilize the defined path iteration methodology, and repeat that for a defined number p of path iterations. The overall pseudocode for our methodology is shown below:

```

for p path iterations do
  Generate random piecewise path
  for n episodes do
    Generate random start-goal pair on path
    Run HRL on path segment
  end for
end for

```

D. Curriculum Learning

Curriculum Learning is a strategy that trains machine learning models by starting with simple examples and then scaling the complexity as the model learns. We attempted to gradually teach our HRL to differentiate which PD controller was ideal for a specific path type by starting with a path composed of two path types, then three, then all four types.

E. Deep Q-Network (DQN) & Double DQN (DDQN)

Our DQN and DDQNs are both implemented with experience replay and epsilon-greedy choice with epsilon decay. The epsilon greedy choice will encourage more randomness with choice of PD controller at the beginning of training so that the model explores all PD controllers in different situations.

The loss for these models is calculated as follows:

$L = \mathbb{E}(Q_{LT} - Q_n)^2$ where Q_{LT} is the Q-Learning Network Target and Q_n is the Q-network

1) *Deep Q-Network*: For DQN, the target function is $Q_{LT} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t^-)$

2) *Double DQN*: For DDQN, the target function is $Q_{LT} = R_{t+1} + \gamma \max_a Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t + t); \theta_t^-)$ [1]

III. IMPLEMENTATION

A. Robot Platform, Simulator, & Computation Power

We simulate a TurtleBot4 robot using a Ros2 workspace and Gazebo simulation. We simulate the TurtleBot on a flat plane and let it follow paths of various curvatures. RL was trained on an M1 Macbook Air with 16GB of memory.

B. PD Controller Tuning

We tuned PD controllers on 4 paths, a straight line and 3 sine curves with identical amplitude but varying frequency.

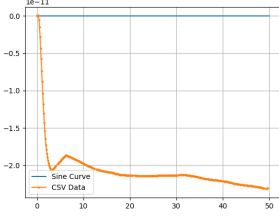
- Straight Line: $y = x$
- Loose Sine Curve: $y = \frac{1}{3} \sin(x)$
- Medium Sine Curve: $y = \frac{1}{3} \sin(3x)$
- Tight Sine Curve: $y = \frac{1}{3} \sin(5x)$

When tuning the PD controllers, each of these base reference paths consist of 5000 points. The points, t , are evenly spaced along a given range, $[0, 10\pi]$. The resulting PD controllers had tuned hyper-parameters k_p and k_d in addition to speed v and horizon h .

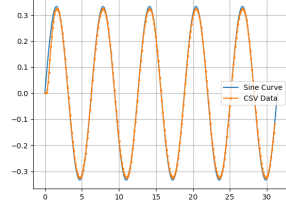
The following are the hyper-parameters in the format of $[k_p, k_d, v, h]$:

- Straight Line: $[1, 0, 1, 100]$
- Loose Sine Curve: $[1.5, 0.001, 1, 100]$
- Medium Sine Curve: $[1.8, 0.001, 0.35, 60]$
- Tight Sine Curve: $[2.5, 0.001, 0.2, 20]$

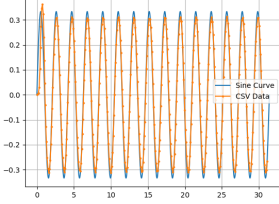
We manually tuned these PD controllers such that the path the TurtleBot traverses matches the reference path. We note that with increasingly tight curves, the k_p increases to react more to the heading error. Additionally, the speed and horizon decrease to make the TurtleBot more nearsighted and react to the sharper turn.



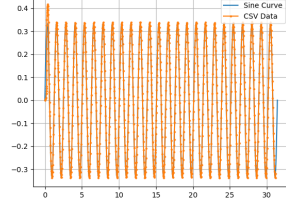
(a) Line PD



(b) Loose Sine PD



(c) Medium Sine PD



(d) Tight Sine PD

Note that for the Line PD controller, it appears to deviate from the line (in blue), but the error is to $1e^{-11}$, indicating only a slight deviation. Additionally, the initial inaccuracy of the robot following the path is due to it compensating for being unaligned with the starting heading.

C. Path Generation

As stated previously, we generated composite paths consisting of our four baseline path types. Each path type's length has its own range, such that for the sine waves of varying frequency, the amount of curve covered in the randomly generated portion is consistent across various types. A composite path consists of around 6 segments for each type of segment in the path. For example, a path consisting of 3 path types would have 18 segments, whereas a path consisting of all 4 paths has 24 segments. The type of each segment in the composed path is random, so the number of segments per path type is not guaranteed to be equal. Additionally, for a generated composite path, 2500 points are allocated for each path type, meaning that if there are 3 path types, then 7500 points are allocated across the generated path.

D. Curriculum Learning

For Curriculum Learning, we first chose two path types to train on and then scale up. When training our models, for each path iteration, we generate a unique random path, and for each episode, we train the RL model on a random subsection 20% of the path. We maintained a pretty consistent training format of 50 episodes per path iteration and aimed to have all four path types in the curriculum learned by the tenth path iteration. This benchmark is based on our previous experience training on two path-types. We attempted various combinations of path types for the levels in the curriculum as follows:

1) Curriculum 1

We wanted to train our HRL in order of paths with increasing curvatures from line to tight sine.

- Stage 1: Path composed of line and loose sine segments

- Stage 2: Path composed of line, loose sine, and mid sine segments

- Stage 3: Path composed of all 4 path types

The training begins with Stage 1 and transitions to Stage 2 on the fourth path iteration, then to Stage 3 on the seventh path iteration.

2) Curriculum 2

We wanted to start with the most differentiable two path-types (line and tight sine) before including more similar path types (loose sine and mid sine).

- Stage 1: Path composed of line and tight sine segments
- Stage 2: Path composed of line, loose sine, and tight sine segments
- Stage 3: Path composed of all 4 path types

The training begins with Stage 1 and transitions to Stage 2 on the fourth path iteration, then to Stage 3 on the seventh path iteration.

3) Curriculum 3

We wanted to teach the HRL to utilize the line PD controller when encountering straight trajectories to optimize for speed. Thus, we added a stage consisting of only line segments.

- Stage 1: Path composed of only line segments
- Stage 2: Path composed of line and mid sine segments
- Stage 3: Path composed of line, loose sine, and mid sine segments
- Stage 4: Path composed of all 4 path types

The training begins with Stage 1 and transitions to Stage 2 on the third path iteration, then to Stage 3 on the fifth path iteration and Stage 4 on the seventh path iteration.

E. Reward Function

We went through 3 path iterations of design for our reward functions, which are as follows.

1) Reward 1

$$R_1 = -2(CTE)^2 - (heading_err)^2 - 0.5\left(\frac{\partial}{\partial t} CTE\right)^2$$

2) Reward 2

We added a positive reward for speed to incentivize our robot not to be overly cautious at the expense of efficiency. $\cos(heading_err)$ decreases the reward when pointing in the wrong direction. This results in the following version 2 reward function:

$$R_2 = -2(CTE)^2 - (heading_err)^2 - 0.5\left(\frac{\partial}{\partial t} CTE\right)^2 + 0.05(speed * \cos(heading_err))$$

3) Reward 3

We removed heading error as it appeared to compound in a way that disincentivized turning, which caused our trained robot to cut corners at the peaks of the tighter sine curves. This resulted in the following version 3 reward function:

$$R_3 = -4(CTE)^2 - 0.5\left(\frac{\partial}{\partial t}CTE\right)^2 + 0.05(speed * \cos(heading_err))$$

Additionally, for all reward functions, if the robot reached the goal, it was rewarded an additional 10 points, meant to incentivize completion.

F. Q-Network

1) *Architecture*: For both the DQN and DDQN Q-Networks, we used the following architecture.

Layer	Type	Size	Activation
Input	State	7	None
1	Linear	256	ReLU
2	Linear	256	ReLU
Output	Linear	256	None

TABLE I: Q Net Architecture

2) *Epsilon-Greedy with Epsilon Decay*: We start with an epsilon of 1, and the epsilon decay factor is 0.99. The minimum possible epsilon value that it can decay to is 0.05. Additionally, when transitioning between different stages in curriculum learning, the epsilon is boosted to 0.2 before continuing to decay.

3) *Target Update*: We utilize a gamma value of 0.9 within our target update function.

G. HRL Training

We tried various training methods and hyper-parameters, but all of our models have a learning rate of 1e-4. Additionally, $k = 5$ and $update_target = 800$ meaning that the learning model is updated every 5 steps and the fixed model is updated to the learning model every 800 steps taken. Each model also trains with max_steps hyper-parameter that is the maximum number of steps that can be taken in a single episode.

Proof of Concept (Milestone 2):

These models were trained to compose the line and medium sine PD controllers to traverse composed paths consisting of line and medium sine segments.

1) DQN: It was trained on 20 path iterations with 50 episodes each (1000 episodes total), and $max_steps = 100$.

This model used Reward 1 which does not account for the speed of the robot.

2) DDQN: It was trained on 10 path iterations with 50 episodes each (500 episodes total), and $max_steps = 150$.

This model used Reward 1 which does not account for the speed of the robot.

Full Implementation:

These models were trained to use the line, loose sine, medium sine, and tight sine PD controllers to traverse composed paths consisting of all four types of line segments.

DDQN with Curriculum Learning

1) Version 1: Curriculum 1 and Reward 2. It was trained on 10 path iterations with 75 episodes each (750 episodes total), and $max_steps = 200$.

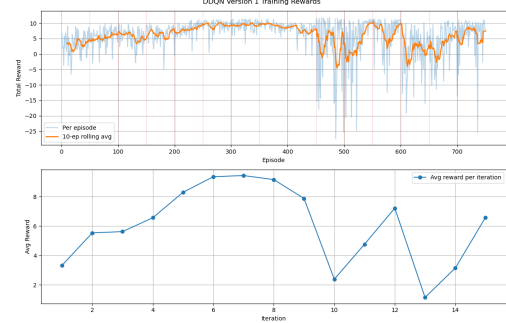


Fig. 2: Learning Curve for Model Version 1

2) Version 2: Curriculum 2 and Reward 2. It was trained on 16 path iterations with 50 episodes each (800 episodes total), and $max_steps = 250$.

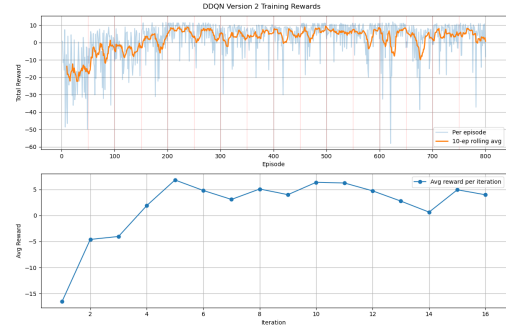


Fig. 3: Learning Curve for Model Version 2

3) Version 3: Curriculum 3 and Reward 2. It was trained on 10 path iterations with 50 episodes each (500 episodes total), and $max_steps = 150$.

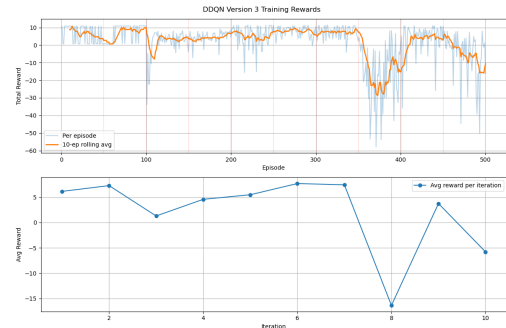


Fig. 4: Learning Curve for Model Version 3

4) Version 4: Curriculum 2 and Reward 3. It was trained on 16 path iterations with 50 episodes each (800 episodes total), and $max_steps = 200$.

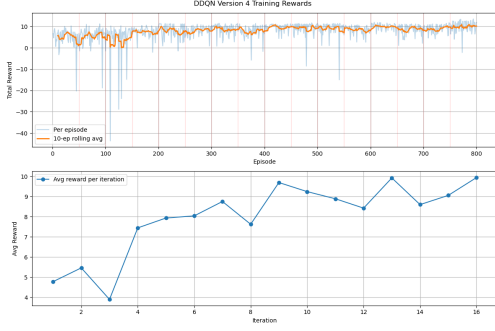


Fig. 5: Learning Curve for Model Version 4

IV. RESULTS

A. Proof of Concept (Milestone 2) Results

We tested our initial DQN and DDQN models trained using line and medium sine PD controllers on paths composed of line and medium sine segments. These were run without limiting the maximum number of steps possible. For each of these runs, the TurtleBot was limited to 1000 steps along the composed path. If it reached the goal, it would earn an additional 10 points for the completion of the course.

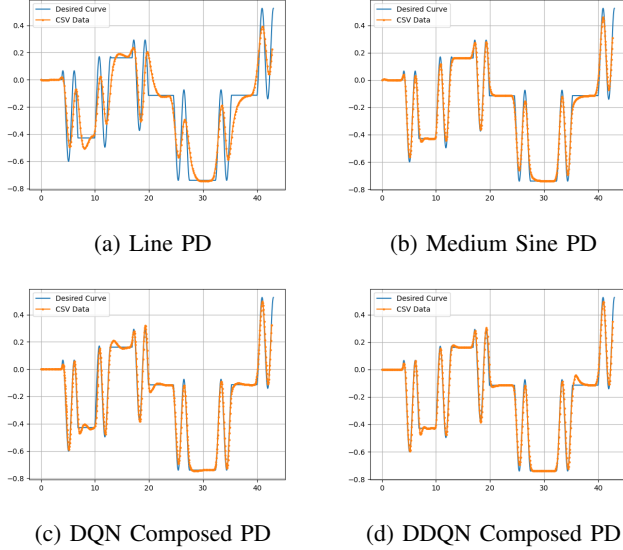


Fig. 6: PD Controller on Line-MidSine Composed Path

We found the DDQN to be more accurate in comparison to the DQN, and very comparable to mid-sine, with it reaching the peaks better.

B. Full Implementation Results

We tested our full implementation on paths composed of the 4 trained on curvatures, as well as variable paths. All path traversals are listed with the reward value calculated using Reward 3.

1) *Composed Paths*: We tested our full implementation models on 2 paths each composed with all 4 curvatures we trained on. The baseline PD controllers that run on the composed paths are shown in Figures 8 and 13. The performance of trained HRL PD controllers in the composed paths is shown in Figures 10 and 15

Composed Path 1

Our first composed path is shown below in Figure 7, with the performance of the PD baseline controllers shown in Figures 8 and 9, which show the robot following the path for each controller and the CTE respectively.

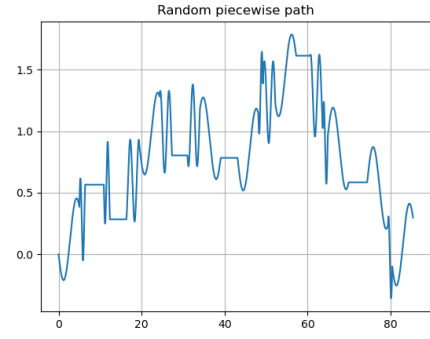


Fig. 7: Composed Path 1

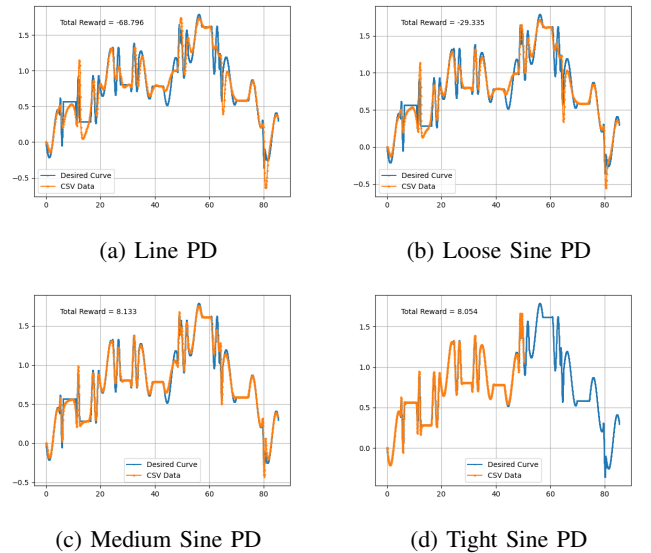
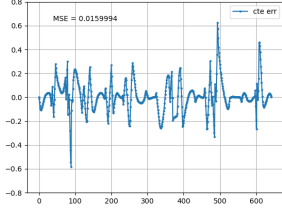
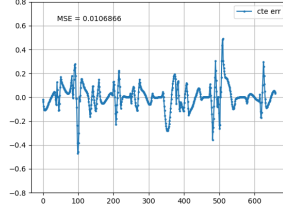


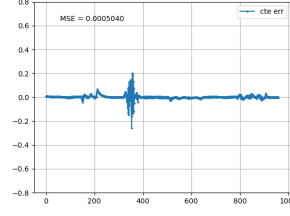
Fig. 8: Baseline PD Controller on Composed Path 1



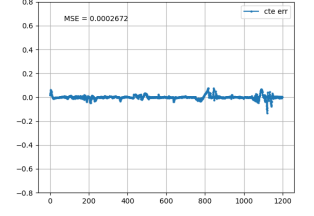
(a) Line PD



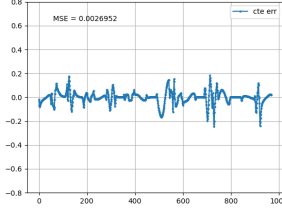
(b) Loose Sine PD



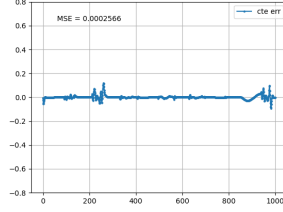
(a) Version 1



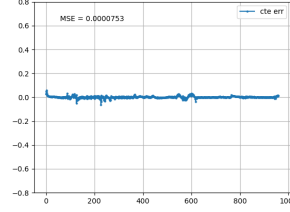
(b) Version 2



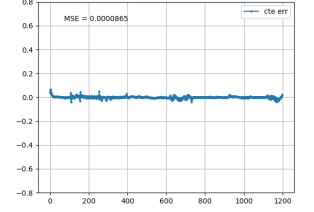
(c) Medium Sine PD



(d) Tight Sine PD



(c) Version 3



(d) Version 4

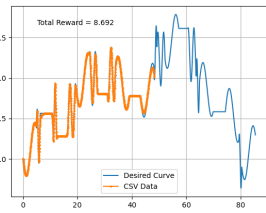
Fig. 9: Baseline PD Controller CTE on Composed Path 1

Fig. 11: HRL PD Controller CTE on Composed Path 1

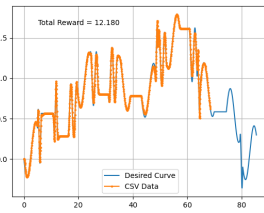
Next, we see the performance of our 4 versions of HRL models shown in Figures 10 and 11, which show the robot following the path for each controller and the CTE respectively.

Composed Path 2

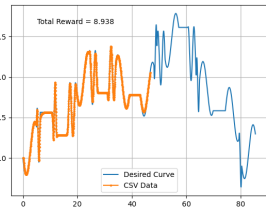
Our second composed path is shown below in Figure 12, with the performance of the PD baseline controllers shown in Figures 13 and 14, which show the robot following the path for each controller and the CTE respectively.



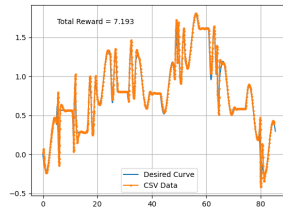
(a) Version 1



(b) Version 2



(c) Version 3



(d) Version 4

Fig. 10: HRL PD Controller on Composed Path 1

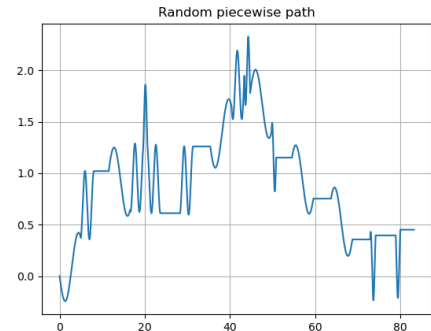
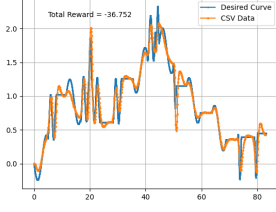
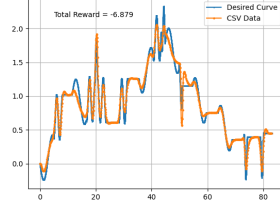


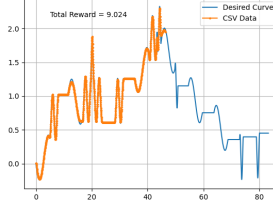
Fig. 12: Composed Path 2



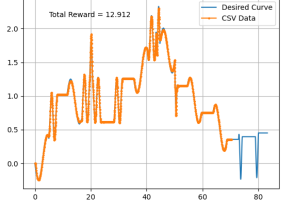
(a) Line PD



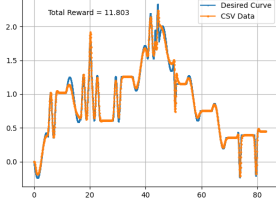
(b) Loose Sine PD



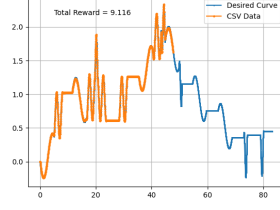
(a) Version 1



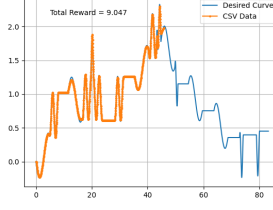
(b) Version 2



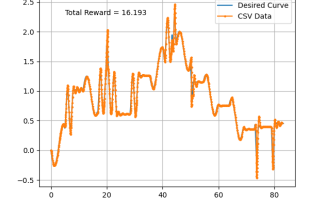
(c) Medium Sine PD



(d) Tight Sine PD



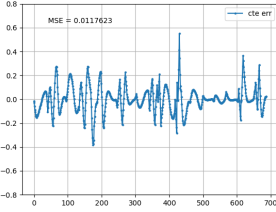
(c) Version 3



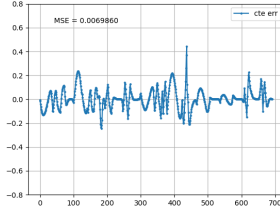
(d) Version 4

Fig. 13: Baseline PD Controller on Composed Path 2

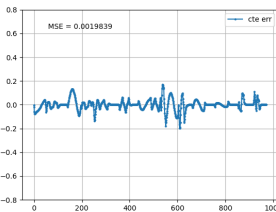
Fig. 15: HRL PD Controller on Composed Path 2



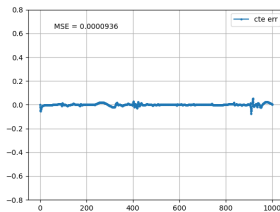
(a) Line PD



(b) Loose Sine PD

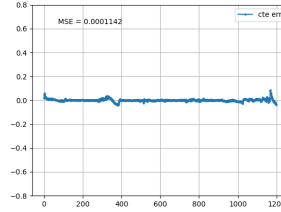


(c) Medium Sine PD

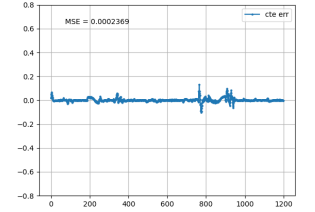


(d) Tight Sine PD

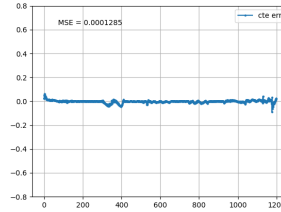
Fig. 14: Baseline PD Controller CTE on Composed Path 2



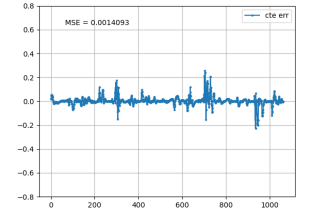
(a) Version 1



(b) Version 2



(c) Version 3



(d) Version 4

Fig. 16: HRL Controller CTE on Composed Path 2

Next, we see the performance of our 4 versions of HRL models shown in Figures 15 and 16, which show the robot following the path for each controller and the CTE respectively.

2) *Variable Paths:* For the full implementation, we tested our model on a circle with radius of $\frac{1}{3}$, a piecewise lemniscate (which we will refer to as the goldfish) in which the right half and left half are different lemniscate functions, and a heart, and we compared these results to the performance of the individual baseline models.

The circle function is

$$\left(\frac{1}{3} \sin(t), \frac{1}{3} \cos(t) + \frac{1}{3}\right) \text{ from } 0 \leq t < 2\pi$$

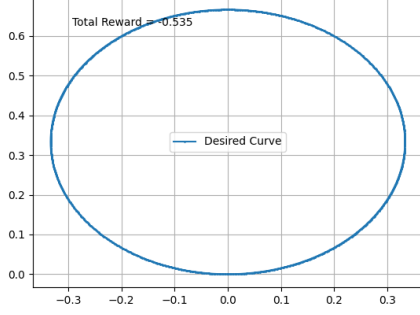


Fig. 17: Circle

The goldfish function is
 $(2 \cos(t - \frac{\pi}{2}), \sin(t) \cos(t))$ for $0 \leq t < \pi$
 and
 $(\cos(t - \frac{\pi}{2}), 2 \sin(t) \cos(t))$ for $\pi \leq t < 2\pi$

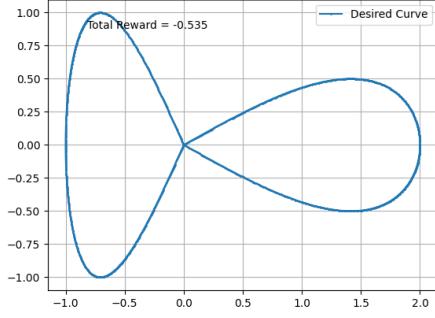


Fig. 18: Goldfish

The heart function is based on a parametric heart from Desmos [2], as follows:

$$(x, y) = ((1.5 \sin t)^3 - 1.7525, -2.8667 + 3 \cos t - 1.2 \cos(2t) - .6 \cos(3t) - .2 \cos(4t))$$

for $0.9335936 \leq t < 7.216778905$

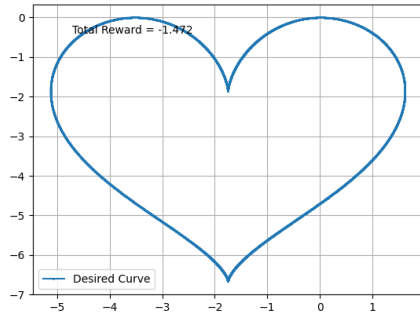


Fig. 19: Heart

For these tests, we gave the robot a set number of steps to take dependent on the path, rather than a specific goal since they loop.

Circle

The steps for circle was 50. Below, we show that the performance of our baselines on the circle in Figures 20 and 21.

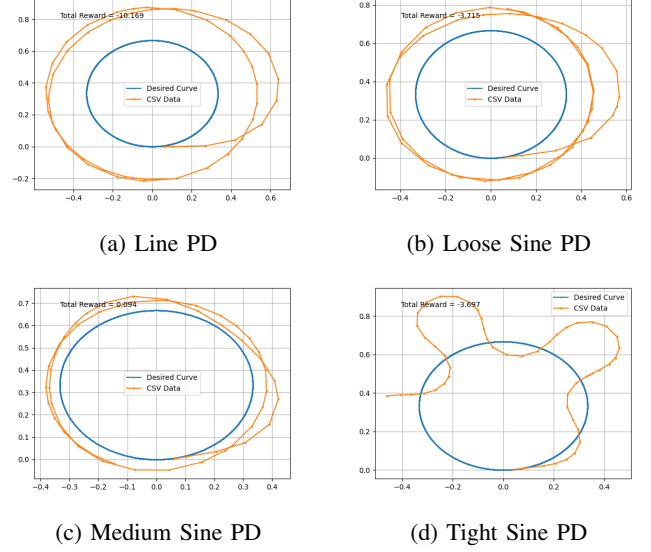


Fig. 20: PD Controller CTE Performance on Circle

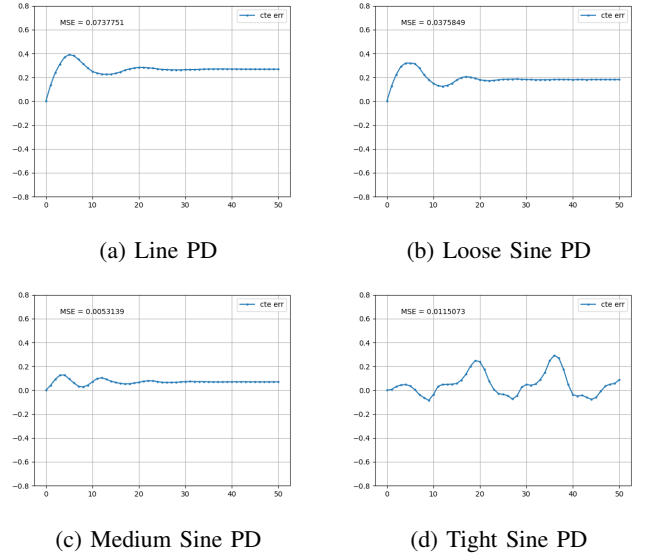


Fig. 21: PD Controller Performance on Circle

Comparatively, we show the performance of Versions 3 and 4 of our DDQN in Figures 22 and 23.

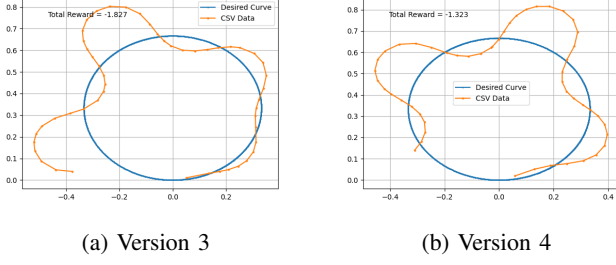


Fig. 22: HRL Controller Performance on Circle

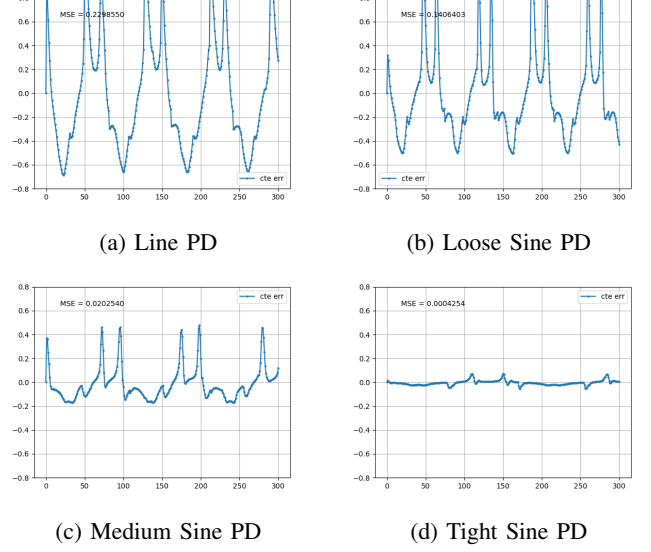


Fig. 25: PD Controller CTE Performance on Goldfish

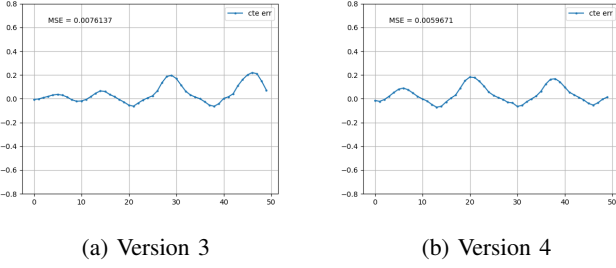


Fig. 23: HRL Controller CTE Performance on Circle

3) *Goldfish*: The steps for our goldfish was 300. Below, we show that the performance of our baselines on the circle in Figures 24 and 25.

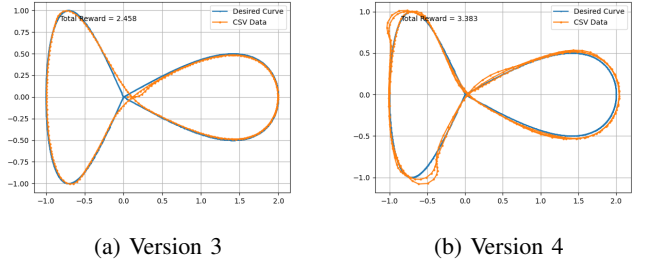


Fig. 26: HRL Controller Performance on Goldfish

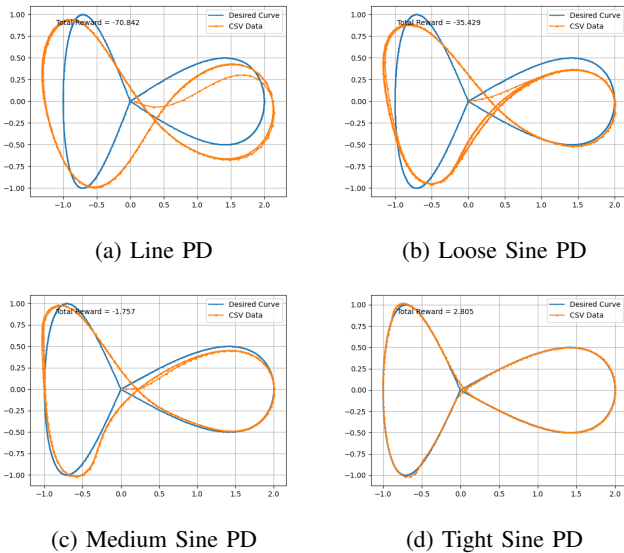


Fig. 24: PD Controller Performance on Goldfish

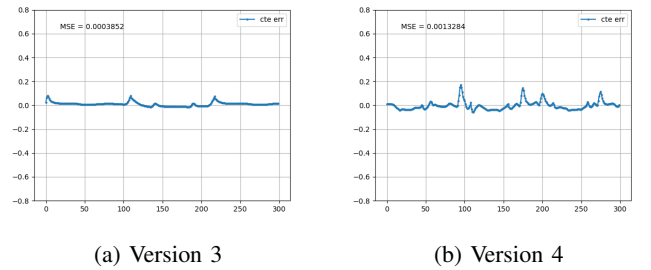


Fig. 27: HRL Controller CTE Performance on Goldfish

4) *Heart*: The steps for our heart was 800 steps. Below, we show that the performance of our baselines on the heart in Figures 28 and 29.

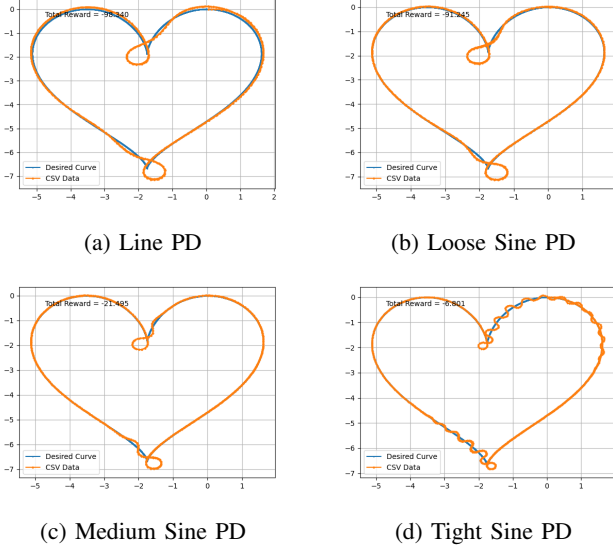


Fig. 28: PD Controller Performance on Heart

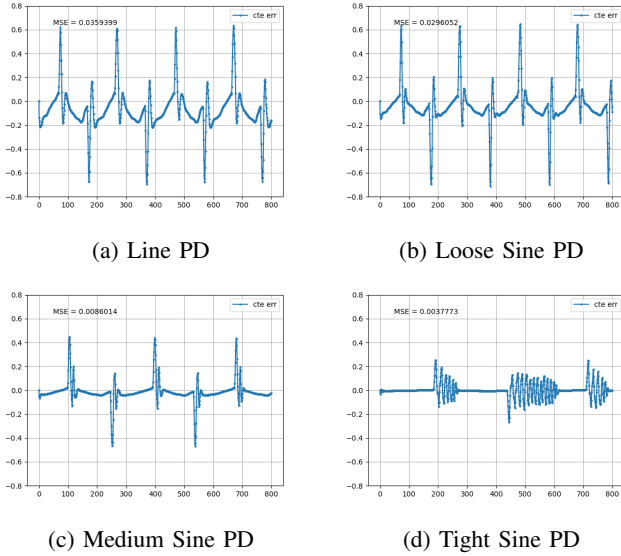


Fig. 29: PD Controller CTE Performance on Heart

Comparatively, we show the performance of Versions 3 and 4 of our DDQN on the heart in Figures 30 and 31.

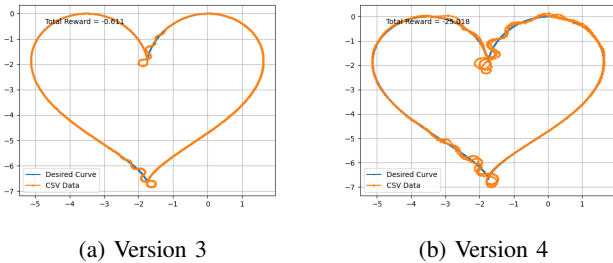


Fig. 30: HRL Controller Performance on Heart

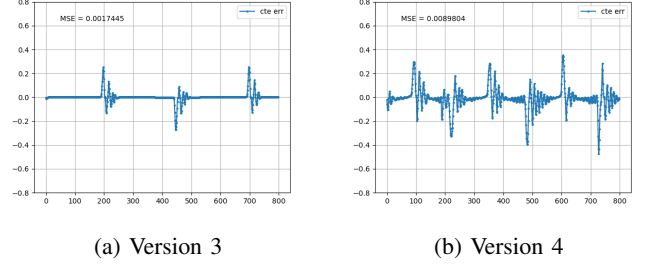


Fig. 31: HRL Controller CTE Performance on Heart

V. DISCUSSION

A. HRL Training and Design

Initially, when scaling from our Milestone 2 HRL model, which was composed using two PD controllers, we attempted to train the HRL model directly on a path composed of all 4 path types. However, this proved ineffective, as after 500 path iterations, it appeared that the model had not learned at all, and the oscillation in the reward was as drastic at the end as it was at the beginning. To enhance the effectiveness of our training, we transitioned to a curriculum-based approach, which begins with simpler training paths before progressing to more complex ones.

There were multiple possible curricula we could train with. Curricula 1 and 3 intend to increase the difficulty over iterations. However, from the observation of the training curve of model versions 1 and 3, it is clear that tight sine curves are difficult to handle, which causes spikes in error and drops in reward later into training 2 4. Eventually, in the interest of accuracy, the model learns to overutilize the tight sine PD controller.

It appears that Curriculum 2, which trained the model to differentiate between line and tight sine curves early on, prevented some of the overutilization of the tight sine PD controllers and allowed the model to optimize using other PD controllers. In turn, this appeared to cause the HRL reward to continue to rise throughout training 3 5.

Furthermore, it was challenging to balance the robot's prioritization of accuracy versus speed. In most situations, the PD controllers line and loose sine are inaccurate, but the advantage of using them is their speed. The goal of composing the PD controllers is for the HRL model to traverse paths more effectively than a single PD controller would. When the composed HRL controller optimizes for accuracy and overutilizes the tight sine PD controller, it progresses very slowly, and its effectiveness is comparable to that of the single tight sine controller.

B. Baseline and HRL Models on Composed Paths

Given enough time, each model would eventually reach the end of the path, but we limited the number of steps the TurtleBot could take to demonstrate the effectiveness of our HRL models and the tradeoff between accuracy and speed. Our reward function (Reward 3) attempts to encapsulate this tradeoff by having a small rewards according to speed and a larger reward when the TurtleBot reaches its goal. Currently,

the reward at the end is 10, however, it may be fruitful for the reward at goal to be proportional to the length of the path.

1) *Baselines*: When observing how the baseline PD controllers performed on the composed paths 8 13, there is a clear tradeoff between accuracy and coverage of the path. The line, loose sine, and medium sine PD controllers all complete the path, while the tight sine PD controller only completes half of the path. When analyzing the corresponding CTE plots 9 14, we see that the error of the tight sine controller is minimized compared to other controllers, which rush the course. Regarding rewards, for Composed Path 1, the medium sine and tight sine PD controllers appear to have the same total reward. This equality implies that completing the course is worth the losses in accuracy, as the tight sine only completed about half of the path. For Composed Path 2, the medium sine PD baseline outperformed the tight sine PD controller due to the reward at the goal, which indicates prioritization of completing the course. Additionally, this discrepancy between the rewards earned on the two paths may imply that Composed Path 2 is harder to traverse than Composed Path 1.

2) *HRL Models*: Upon analyzing the traversal of our various HRL models on composed paths 10 15, it's immediately apparent that only one model, Version 4, completes the course. Version 2 gets further than Versions 1 and 3, which make the least progress but are the most accurate 11 16. Furthermore, when referencing the rewards of each model, no model consistently scores the highest. However, versions 2 and 4 seem to score pretty highly. For Composed Path 1, model version 4 is outscored by versions 1 and 3; however, those two models did not reach the section of the path with the sharpest transitions, which may have negatively impacted the reward of version 4. On another note, when model version four traverses a composed path, it appears not to utilize the line PD controller from the beginning, as evidenced by the squiggle when transitioning from the previous curve into the line. This suboptimal behavior may be due to a possible design flaw in our composed paths. When the HRL detects a transition between curves, it selects a controller, but the line segments are not long enough to properly learn to utilize the line PD controller.

3) *Baselines vs HRL Models*: Comparing our HRL models with the baseline controller traversals, model versions 1 and 3 perform on par with the tight sine controller, whereas model version 2 outperforms medium sine in terms of accuracy but not path coverage. Additionally, model version 4 is the only HRL model to complete the path within 1000 steps and appears to have an accuracy that matches or outperforms the medium sine PD controller. Regarding the rewards, our HRL models outperform any individual baseline controller. Therefore, after comparing the HRL models to each other and to the baselines, when traversing a composed path, model version 3 has the highest accuracy, but model versions 2 and 4 are more efficient overall.

C. Baseline and HRL Model on Variable Paths

On the variable paths, we focused on testing versions 3 and 4 on the variable paths in comparison to the baseline models,

since, as discussed, these were our best performing models in means of accuracy for one, and speed for the other.

1) *Circle*: For the circle, none of the baselines seemed to perform very well, but as seen in Figures 20 and 21, the Medium Sine PD controller followed the path the most closely and had the least error. The Tight Sine PD controller appeared to oscillate the most, and was the most similar in behavior that we saw in both our DDQNs, as seen in figure 22.

2) *Goldfish*: Our goldfish exhibits some interesting behavior for the controllers. As seen in Figures 24 and 25, the tight sine is the most accurate, but it is also our slowest controller, as it only goes around the goldfish 1.5 times. In comparison, the Line and Loose Sine go closer to 2.5 times around, so it traverses the path less times in comparison to the other PD Controllers. However, since it is much more accurate, it does have a higher reward, where it's reward is 2.8. As seen in Figures 26 and 27, our version 3 appears to perform very similarly to Tight Sine, with fairly comparable accuracy and number of iterations it traversed, but reward-wise performed less well than Tight Sine. Our Version 4 DDQN outperforms Tight Sine by our reward function, while also traversing more iterations of the path than the tight sin controller did, as it goes upwards of 2 times around the path, indicating successful switching of controllers within our trained model on this variable path. It is not as accurate as the Tight Sine, but does make up for that in speed, while also having better error than all baseline PD controllers other than Tight Sine.

3) *Heart*: For the heart, there appears to be some similar behavior as goldfish. Firstly, let's discuss the PD Controller performance on the heart. By precision, it can be seen in figures 28 and 29, that the Medium and Tight Sine are our strongest PD Controllers for this shape. Both do have lower speed than the Line and Loose Sin, so that is to be expected, but it is evident from the goldfish and heart CTE graphs that compared to the other CTEs, tight sin appears to have the greatest precision, closely followed by Medium Sine. However, on this task, the speed difference was very evident, with Tight Sine only running a little more than an iteration and a half of the heart, while the Medium Sine did about 2 and a half iterations. In contrast, the Line and Loose Sin did 4 iterations each.

In comparison, our version 3 performed very similarly to Tight Sine, with only an iteration and a half of the heart ran and very similar accuracy. Despite these similarities, it appears that, as seen in Figures 30 and 31, the robot appears to return to following the path quicker than it did in the Tight Sine after the points at the bottom and top of the heart.

In contrast, as seen in Figures 30 and 31, our version 4 does have more error and deviate from the path more than the Tight Sine and Medium Sine, it does perform more iterations than both around the heart, as it goes 3 times around the heart. Interestingly, it appeared to take longer than most of the PD controllers to return to the paths after the points at the top and bottom of the heart, but generally has less error than all the PD controllers except for Tight Sine. Due to it's greater oscillation, it performed slightly worse than Medium Sine, but significantly better than Line and Loose Sine.

D. Overall Performance on Variable Paths

From the analyzed performance on the variable paths, it appears that of the tested models, our fourth version appears to tradeoff the speed and accuracy the best in comparison to the other DDQNs, and the performance of the baseline PD controllers.

The circle was an incredibly tight radius circle, and given that with any immediate error, we hypothesize the model might gravitate toward utilizing the Tight Sine PD Controller, it makes sense that it might appear to utilize the tight sin to correct, but reward-wise, it performed only slightly worse than the best PD controller, which was Medium Sine. The goldfish is where the potential of this model became much more evident. As discussed, the version 4 also is very accurate for much of the goldfish path, while being much faster than the Tight Sine PD Controller, and this holds for the heart as well.

Therefore, we believe our fourth version shows much potential for generalizability to more variable paths, with comparable accuracy to the most accurate PD controller that we trained, the Tight Sine, but greater speed.

VI. CONCLUSION

To summarize, we utilized curriculum learning-based DDQN to train various HRL-composed PD controllers from four original curvature-tuned PD controllers. The goal was to create a model that selects the best controller configuration for a given subtask, a segment of an overarching path. Of our four models, we believe model version 4 performs the best overall, particularly because it exhibits accuracy comparable to the tight sine PD controller while traveling faster through the path.

VII. STATEMENT OF CONTRIBUTION

Mahima Malhotra

- Environment/Simulation Setup
- PD Controller Code
- Reinforcement Learning Code (Policy Gradient, DQN, DDQN)
- Error Calculation and Visualization
- Reward Function Debugging and Tuning
- Variable Paths Construction and Related Visualizations
- Analysis of Baseline Controllers and HRL models on Variable Paths

Jiahui Huang

- PD Controller Tuning and Visualization
- Random Composed Path Construction
- Start/Goal Point Configuration Logic
- Curriculum Learning
- Reward Function Construction and Tuning
- DQN & DDQN Training and Related Visualizations
- Analysis of Baseline Controllers and HRL models on Composed Paths

REFERENCES

- [1] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” 2015. [Online]. Available: <https://arxiv.org/abs/1509.06461>
- [2] [Online]. Available: <https://www.desmos.com/calculator/5o30ytt6sg>